

890. Find and Replace Pattern

Medium Topics Companies

Given a list of strings `words` and a string `pattern`, return a list of `words[i]` that match `pattern`. You may return the answer in **any order**.

A word matches the pattern if there exists a permutation of letters `p` so that after replacing every letter `x` in the pattern with `p(x)`, we get the desired word.

Recall that a permutation of letters is a bijection from letters to letters: every letter maps to another letter, and no two letters map to the same letter.

Example 1:

Input: `words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"`

Output: `["mee","aqq"]`

Explanation: "mee" matches the pattern because there is a permutation `{a -> m, b -> e, ...}`.
"ccc" does not match the pattern because `{a -> c, b -> c, ...}` is not a permutation, since a and b map to the same letter.

Example 2:

Input: `words = ["a","b","c"], pattern = "a"`

Output: `["a","b","c"]`

Constraints:

- `1 <= pattern.length <= 20`
- `1 <= words.length <= 50`
- `words[i].length == pattern.length`
- `pattern` and `words[i]` are lowercase English letters.



We built the AI, they got the A.

[Learn More](#)

Seen this question in a real interview before? 1/6

4.1K 28 ☆ ↺

3 Online

```

1
2 import java.util.*;
3
4 class Solution {
5
6     public List<String> findAndReplacePattern(String[] words, String pattern) {
7
8         List<String> result = new ArrayList<>();
9
10        for (String word : words) {
11
12            if (matches(word, pattern)) {
13                result.add(word);
14            }
15        }
16
17        return result;
18    }
19
20    private boolean matches(String word, String pattern) {
21
22        char[] map1 = new char[26];
23        char[] map2 = new char[26];
24
25        for (int i = 0; i < pattern.length(); i++) {
26
27            int p = pattern.charAt(i) - 'a';
28            int w = word.charAt(i) - 'a';
29
30            // Pattern character already has a mapping
31            if (map1[p] != 0 && map1[p] != word.charAt(i)) {
32                return false;
33            }
34
35            // Word character is already mapped to another pattern character
36            if (map2[w] != 0 && map2[w] != pattern.charAt(i)) {
37                return false;
38            }
39
40            map1[p] = word.charAt(i);
41            map2[w] = pattern.charAt(i);
42        }
43
44        return true;
45    }
46 }
47
48

```

Saved

Ln 47, Col 1

Testcase Test Result

890. Find and Replace Pattern

Medium Topics Companies

Given a list of strings `words` and a string `pattern`, return *a list of `words[i]` that match `pattern`*. You may return the answer in **any order**.

A word matches the pattern if there exists a permutation of letters `p` so that after replacing every letter `x` in the pattern with `p(x)`, we get the desired word.

Recall that a permutation of letters is a bijection from letters to letters: every letter maps to another letter, and no two letters map to the same letter.

Example 1:

Input: `words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"`

Output: `["mee","aqq"]`

Explanation: "mee" matches the pattern because there is a permutation `{a -> m, b -> e, ...}`.
"ccc" does not match the pattern because `{a -> c, b -> c, ...}` is not a permutation, since a and b map to the same letter.

Example 2:

Input: `words = ["a","b","c"], pattern = "a"`

Output: `["a","b","c"]`

Constraints:

- `1 <= pattern.length <= 20`
- `1 <= words.length <= 50`
- `words[i].length == pattern.length`
- `pattern` and `words[i]` are lowercase English letters.

4.1K 28

4 Online

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`words =`
`["abc","deq","mee","aqq","dkd","ccc"]`

`pattern =`
`"abb"`

Output

`["mee","aqq"]`

Expected

`["mee","aqq"]`

Contribute a testcase